
Clase 121 — Custom training loops (+ PyTorch & PyTorch Lightning)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 12 § Custom Training Loops + docs PyTorch, Lightning. Duración estimada: 85 min. · Nivel: Avanzado

Clase 121 — Custom training loops (+ PyTorch & PyTorch Lightning)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 12 § Custom Training Loops + docs PyTorch, Lightning.

Duración estimada: 85 min. · Nivel: Avanzado

Objetivo

Escribir un training loop manual en TF con GradientTape — control absoluto sobre cada paso (útil para GANs, RL, multi-step optimizers, debugging). Conocer el equivalente en PyTorch (el framework dominante de la industria en 2026) y cómo PyTorch Lightning abstrae los boilerplate del loop, devolviendo la productividad de Keras con la flexibilidad de PyTorch.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Escribir un training loop TF con `for batch in dataset: with GradientTape() as tape: ...; grads = tape.gradient(loss, vars); optimizer.apply_gradients(...)`.
- Hacer el equivalente en PyTorch: `optimizer.zero_grad(); loss.backward(); optimizer.step()`.
- Usar PyTorch Lightning para el mismo problema con boilerplate mínimo (`LightningModule.training_step`).
- Reconocer cuándo el loop manual es necesario (modelo con dos optimizadores, schedule custom step-wise, RL).
- Comparar productividad y flexibilidad de los 3 frameworks.

Temas

- Loop básico TF: `epochs` → `batches` → `tape` → `grads` → `apply`.
- `tape.watch(...)` para watch tensors que no son `tf.Variable`.
- Métricas y logging manual.
- Equivalente PyTorch.
- Lightning como capa de abstracción.
- Complemento moderno: PyTorch + Lightning en paralelo a TF/Keras.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 122 — PyTorch fundamentos: tensores, autograd, `nn.Module`
- Clase 108b — PyTorch Lightning: Trainer + distributed

Definiciones y características

- GradientTape: contexto TF que registra operaciones para autodiff.

- `tape.watch(t)`: forzar a tape a registrar un tensor no-Variable.
- `tape.gradient(loss, vars)`: calcular gradientes.
- `optimizer.apply_gradients(zip(grads, vars))`: aplicar la update.
- `PyTorch .backward()`: aplica autodiff sobre el grafo dinámico construido en el forward.
- `requires_grad`: flag PyTorch que indica si un tensor necesita gradientes.

Dataset / recursos

- Fashion-MNIST o MNIST.
- Librerías: tensorflow, torch, lightning (pip install lightning).

Ejercicios

1. TF loop manual: entrenar un MLP en Fashion-MNIST con GradientTape. Implementar logging de loss y métrica manual.
2. PyTorch equivalente: reimplementar el mismo loop en PyTorch.
3. Lightning: mismo problema con LightningModule.
4. Speedup con jit: `tf.function` en TF; `torch.compile` en PyTorch. Medir.
5. Multi-optimizer: con TF, escribir un loop que aplica un optimizer para las capas frozen-ish (LR bajo) y otro para las nuevas (LR alto). Esto en `model.fit` requiere mucho boilerplate.

Homework verifiable

Implementar el mismo problema (Fashion-MNIST, MLP [256, 128]) en los 3 frameworks:

1. TF con custom training loop.
2. PyTorch puro.
3. PyTorch Lightning.

Reportar para cada uno: # líneas de código, tiempo de training, accuracy.

Criterio de aceptación: las 3 versiones llegan a accuracy similar (± 0.5 pp). Lightning $\approx$ Keras en LOC; PyTorch puro tiene más boilerplate; TF custom loop también.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Loss no baja en TF custom loop	Olvidaste <code>tape.gradient(loss, model.trainable_variables)</code>
En PyTorch, gradientes acumulados batch a batch	Olvidaste <code>optimizer.zero_grad()</code> al inicio
<code>RuntimeError: cudnn error</code> en PyTorch	Versión de PyTorch incompatible con CUDA i
Lightning espera <code>LightningDataModule</code> cuando llamas <code>trainer.fit</code>	Lightning acepta ambos; verificar firma de <code>LightningDataModule</code>
BatchNorm no se actualiza en TF custom loop	Hay que pasar <code>training=True</code> explícitamente

Preguntas frecuentes

¿TF o PyTorch para empezar?

Keras (TF) es más amigable para empezar. PyTorch es más demandado en producción 2026. Saber ambos

es el estándar profesional.

¿Lightning vs Keras de cuál sale mejor?

Lightning te abre el ecosistema PyTorch (Hugging Face). Keras 3 ahora soporta backends multi (TF, JAX, PyTorch) — la diferencia disminuye.

¿zero_grad() por qué existe en PyTorch?

Para permitir gradient accumulation (varios .backward() sin step para batches grandes virtuales). Si no necesitás eso, llamá zero_grad() al inicio del batch siempre.

¿Distributed training más fácil en cuál?

Lightning, por mucho. `trainer = L.Trainer(strategy='ddp', devices=4)` y ya. En TF necesitás `tf.distribute.MirroredStrategy` con `strategy.scope():`. PyTorch puro requiere `torch.distributed.init_process_group` manual.

¿Hugging Face usa PyTorch o TF?

Casi todo PyTorch desde 2022. Los modelos modernos (Llama, Mistral, Mixtral, etc.) están solo en PyTorch.

Referencias

- Géron, cap. 12 — Custom Training Loops.
- PyTorch tutorials.
- PyTorch Lightning docs.
- Falcon et al. (2019), PyTorch Lightning.
- Hugging Face — Trainer (built on Lightning concepts).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Siguiente clase

Clase 122 — PyTorch fundamentos: tensores, autograd, nn.Module

Apéndice: notebook (primer bloque)

Primera celda ejecutable del notebook de la clase.

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
keras.utils.set_random_seed(42)

(X_tr, y_tr), (X_te, y_te) = keras.datasets.fashion_mnist.load_data()
X_tr = X_tr.reshape(-1, 784).astype("float32") / 255.0
X_te = X_te.reshape(-1, 784).astype("float32") / 255.0
```

```
train_ds = (tf.data.Dataset.from_tensor_slices((X_tr, y_tr))
            .shuffle(1024).batch(128).prefetch(tf.data.AUTOTUNE))

modelo = keras.Sequential([
    keras.Input((784,)),
    layers.Dense(256, activation="relu"),
    layers.Dense(128, activation="relu"),
    layers.Dense(10,          # logits (sin softmax)
)]
print("batches por época:", len(train_ds))
```

Archivos complementarios

- notebook.ipynb